

Specifications

Specifications

PredicateSpecification

Specification, UpdateSpecification, DeleteSpecification

Fluent API

JPA's Criteria API lets you build queries programmatically. Spring Data JPA `Specification` provides a small, focused API to express predicates over entities and reuse them across repositories. Based on the concept of a specification from Eric Evans' book "Domain Driven Design", specifications follow the same semantics providing an API to define criteria using JPA. To support specifications, you can extend your repository interface with the `JpaSpecificationExecutor` interface, as follows:

```
public interface CustomerRepository extends CrudRepository<Customer, Long>,
    JpaSpecificationExecutor<Customer> {
}
```

JAVA

A specification is a predicate over an entity expressed with the Criteria API. Spring Data JPA offers two entry points:

- [PredicateSpecification](#): A flexible, query-type-agnostic interface introduced with Spring Data JPA 4.0.
- [Specification](#) (and `UpdateSpecification`, `DeleteSpecification`): Query-bound variants.

PredicateSpecification

The `PredicateSpecification` interface is defined with a minimal set of dependencies allowing broad functional composition:

```
public interface PredicateSpecification<T> {
    Predicate toPredicate(From<?, T> from, CriteriaBuilder builder);
}
```

JAVA

Specifications can easily be used to build an extensible set of predicates and used with `JpaRepository` removing the need to declare a query (method) for every needed combination as shown in the following example:

Example 1. Specifications for a Customer

```
class CustomerSpecs {  
  
    static PredicateSpecification<Customer> isLongTermCustomer() {  
        return (from, builder) -> {  
            LocalDate date = LocalDate.now().minusYears(2);  
            return builder.lessThan(from.get(Customer_.createdAt), date);  
        };  
    }  
  
    static PredicateSpecification<Customer> hasSalesOfMoreThan(MonetaryAmount value) {  
        return (from, builder) -> {  
            // build predicate for sales > value  
        };  
    }  
}
```

JAVA

The `Customer_` type is a metamodel type generated using the JPA Metamodel generator (see the [Hibernate implementation's documentation for an example](#)). So the expression, `Customer_.createdAt`, assumes the `Customer` has a `createdAt` attribute of type `Date`. Besides that, we have expressed some criteria on a business requirement abstraction level and created executable `Specifications`.

Use a specification directly with a repository:

```
List<Customer> customers = customerRepository.findAll(isLongTermCustomer());
```

JAVA

Specifications become most valuable when composed:

```
MonetaryAmount amount = new MonetaryAmount(200.0, Currencies.DOLLAR);  
List<Customer> customers = customerRepository.findAll(  
    isLongTermCustomer().or(hasSalesOfMoreThan(amount))  
);
```

JAVA

Specification, UpdateSpecification, DeleteSpecification

The [Specification](#) interface has been available for a much longer time and is tied a particular query type (select, update, delete) as per Criteria API restrictions. The three specification interfaces are defined as follows:

Specification	UpdateSpecification	DeleteSpecification
---------------	---------------------	---------------------

```
public interface Specification<T> {  
    Predicate toPredicate(Root<T> root, CriteriaQuery<?> query,  
        CriteriaBuilder builder);  
}
```

JAVA

Specification objects can be constructed either directly or by reusing PredicateSpecification instances, as shown in the following example:

Example 2. Specifications for a Customer

```
public class CustomerSpecs {  
  
    public static UpdateSpecification<Customer>  
updateLastnameByFirstnameAndLastname(String newLastName, String currentFirstname,  
String currentLastname) {  
        return UpdateSpecification<User> updateLastname = UpdateSpecification.<User>  
update((root, update, criteriaBuilder) -> {  
            update.set("lastname", newLastName);  
        }).where(hasFirstname(currentFirstname).and(hasLastname(currentLastname)));  
    }  
  
    public static PredicateSpecification<Customer> hasFirstname(String firstname) {  
        return (root, builder) -> {  
            return builder.equal(from.get("firstname"), value);  
        };  
    }  
  
    public static PredicateSpecification<Customer> hasLastname(String lastname) {  
        return (root, builder) -> {  
            // build query here  
        };  
    }  
}
```

JAVA

Fluent API

`JpaSpecificationExecutor` defines fluent query methods for flexible execution of queries based on `Specification` instances:

1. For `PredicateSpecification`: `findBy(PredicateSpecification<T> spec, Function<? super SpecificationFluentQuery<S>, R> queryFunction)`
2. For `Specification`: `findBy(Specification<T> spec, Function<? super SpecificationFluentQuery<S>, R> queryFunction)`

As with other methods, it executes a query derived from a `Specification`. However, the query function allows you to take control over aspects of query execution that you cannot dynamically control otherwise. You do so by invoking the various intermediate and terminal methods of `SpecificationFluentQuery`.

Intermediate methods

- `sortBy`: Apply an ordering for your result. Repeated method calls append each `Sort` (note that `page(Pageable)` using a sorted `Pageable` overrides any previous sort order).
- `limit`: Limit the result count.
- `as`: Specify the type to be read or projected to.
- `project`: Limit the queries properties.

Terminal methods

- `first`, `firstValue`: Return the first value. `first` returns an `Optional<T>` or `Optional.empty()` if the query did not yield any result. `firstValue` is its nullable variant without the need to use `Optional`.
- `one`, `oneValue`: Return the one value. `one` returns an `Optional<T>` or `Optional.empty()` if the query did not yield any result. `oneValue` is its nullable variant without the need to use `Optional`. Throws `IncorrectResultSizeDataAccessException` if more than one match found.
- `all`: Return all results as a `List<T>`.
- `page(Pageable)`: Return all results as a `Page<T>`.
- `slice(Pageable)`: Return all results as a `Slice<T>`.
- `scroll(ScrollPosition)`: Use scrolling (offset, keyset) to retrieve results as a `Window<T>`.
- `stream()`: Return a `Stream<T>` to process results lazily. The stream is stateful and must be closed after use.
- `count` and `exists`: Return the count of matching entities or whether any match exists.

Intermediate and terminal methods must be invoked within the query function.

Example 3. Use the fluent API to get a projected Page, ordered by Lastname

```
Page<CustomerProjection> page = repository.findBy(spec,  
    q -> q.as(CustomerProjection.class)  
        .page(PageRequest.of(0, 20, Sort.by("lastname")))  
);
```

JAVA

Example 4. Use the fluent API to get the last of potentially many results, ordered by Lastname

```
Optional<Customer> match = repository.findBy(spec,  
    q -> q.sortBy(Sort.by("lastname").descending())  
        .first()  
);
```

JAVA



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.